Consigna

UnoArena: Global Real-Time Uno Platform & Massive Tournaments

The Problem: Build the backend for a highly competitive Uno platform that supports both individual, ad-hoc game rooms (2-10 players) and massive, multi-tiered elimination tournaments (up to 1,000,000 players). Actions are submitted via a REST API, while players and spectators receive simultaneous state updates via Server-Sent Events (SSE). The system must handle split-second game mechanics (like calling "Uno!"), real-time tournament bracket progression, and a global Elo-based ranking system.

How to play Uno: https://www.youtube.com/watch?v=rC-DYC3ZELM (https://www.youtube.com/watch?v=rC-DYC3ZELM)

Core Engineering Challenges:

- **Hierarchical Lifecycle Management (Rooms & Tournaments):** Managing the state machine at two distinct levels. Individual rooms transition through explicit Kafka events (waiting → in_progress → completed) and spin up as dedicated state-machine pods in Kubernetes. Above this, a Tournament Orchestrator service manages brackets for 1M players, handling the "thundering herd" problem of spinning up 100,000+ concurrent game pods for the first round. room.completed events trigger a saga that securely reports scores, advances winners, and orchestrates the next tournament phase.
- **Authoritative Deck & RNG Service:** All card shuffling and draws are generated server-side by a dedicated, seeded RNG service. Every single state change (e.g., card.played, color.changed, penalty.drawn) is appended to an immutable game log before being broadcast. This makes every random outcome auditable, replay-safe, and highly secure—which is absolutely critical for dispute resolution in cash-prize or high-stakes tournament tiers.
- **REST/SSE Fan-Out Architecture:** Clients submit their moves via standard REST endpoints (e.g., POST /rooms/{id}/play), but state updates are strictly unidirectional via SSE. Each room-state pod publishes delta patches to a Redis Streams channel. A massive, stateless SSE broadcaster tier—scaled independently to handle hundreds of thousands of long-lived HTTP connections—consumes those patches and pushes them to clients. This prevents connection-heavy SSE streams from overwhelming the CPU-bound game logic pods.
- **Strict Concurrency & Reactive Rules Enforcement:** Uno requires handling split-second, highly concurrent actions (e.g., stacking +2 cards, jump-ins, or the race to call "Uno!"

before another player spots you). The room-state service must serialize concurrent REST POST requests using strict sequence numbers. If a player submits an action against a stale game state (e.g., someone reversed the order a millisecond prior), the API instantly rejects it with an HTTP 409 Conflict, leveraging a clean client UX contract to automatically reconcile with the incoming SSE stream.

- **Tournament Analytics & Bracket CQRS:** The platform must survive the massive spike of game.completed events that fire off simultaneously at the end of a tournament round. A pure read model built via CQRS consumes these events across the Kafka bus, projecting per-player statistics (win rate, Elo change) and updating massive, denormalized tournament bracket visualizations in a highly available store (like Redis or DynamoDB) optimized purely for heavy read traffic.

What makes it hard at 1M users: The primary bottleneck is the synchronization of massive tournament rounds combined with SSE connection limits. When a 1M-player tournament starts, the backend must instantly provision resources for over 100,000 parallel games. Furthermore, maintaining 1M open HTTP connections for SSE requires massive OS-level tuning (file descriptors, ephemeral ports). The separation of the room-state pods (heavy, stateful game logic) from the SSE broadcaster tier (lightweight, stateless connection management) is the critical architectural insight required to survive tournament load spikes without dropping game state.